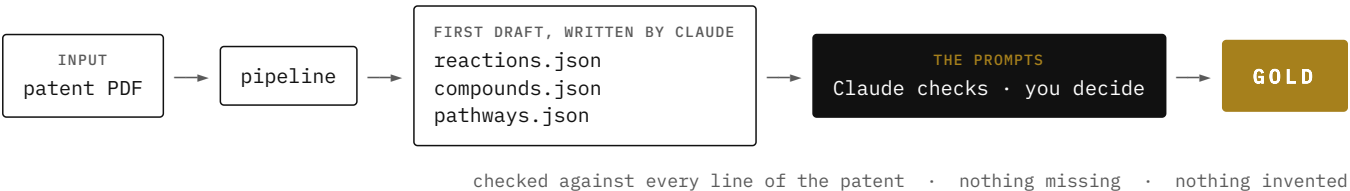


Golden Patents Manual

We check the pipeline's output for one patent until it is provably right. That checked output is the **golden dataset**, and it is what the pipeline is measured against.



YOU NEED

the repo, Claude Code, the prompts in prompts/

YOU GET

gold JSON for one patent, plus an audit trail

CLAUDE DOES

reads the patent, flags what is wrong

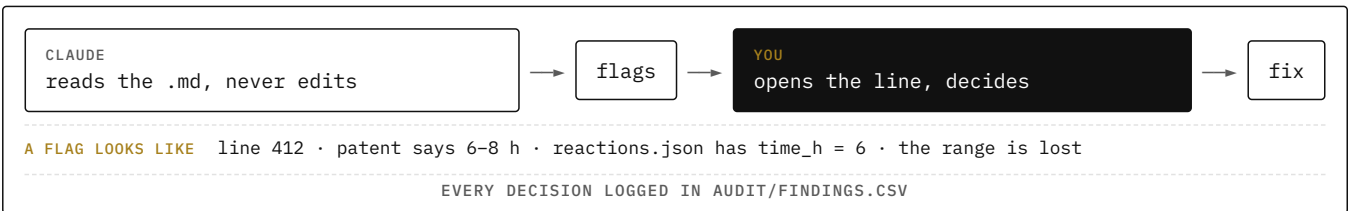
YOU DO

decide every flag, then order the fix

SECTION 2

Your role

Claude reads and flags. You decide and fix. Nothing changes in the gold until you say so.



SECTION 3

Before you start

1 Get the repo

```
git clone https://github.com/yashmstack/patent-extraction
cd patent-extraction
```

2 Run your patent

Follow the repo README. In short:

```
claude
/run-pipeline <PATENT_ID>          # e.g. W02024109718A1
```

When it finishes you have `runs/<PATENT_ID>/` with the patent, the markdown Claude reads, and the extracted JSON.

3 Make the gold folder

```
goldenPatents/<PATENT_ID>/
  input/      <ID>.pdf
              <ID>-enriched-numbered.md    what Claude reads
  output/     reactions.json  compounds.json  pathways.json
  audit/      created by the prompts
```

```
ID=<PATENT_ID>
mkdir -p goldenPatents/$ID/{input,output}
cp runs/$ID/input/pdf/*.pdf                goldenPatents/$ID/input/
cp runs/$ID/input/$ID-enriched-numbered.md  goldenPatents/$ID/input/
cp runs/$ID/output/{reactions,compounds,pathways}.json goldenPatents/$ID/output/
```

4 Open Claude Code there and run the prompts

```
cd goldenPatents/<PATENT_ID>
claude
```

Paste the prompts from `prompts/` in number order.

Claude reads the `.md`, not the PDF: it is translated to English and line-numbered, so every value in the gold points back to a line.

SECTION 4

The passes, at a glance



SECTION 5

The prompts, one by one

1 Identify reactions

REACTIONS · FLAGS

Reads the `.md` alone, never the JSON, and counts every reaction and step, with line numbers.

→ your own list and count

3 Identify compounds

COMPOUNDS · FLAGS

Reads the `.md` alone and lists every compound the patent names, with line numbers.

→ your own list and count

5 Verify reactions

REACTIONS · FLAGS

One record at a time, the whole `.md` re-read for each, every field checked.

→ reactions_audit.csv

7 Gold check reactions

REACTIONS · EDITS

Re-derives every field, corrects as it goes, merges duplicates, gives a verdict on every finding.

→ corrected file · yes / no

9 Adversarial audit

ALL FILES · FLAGS

Fresh agents with no context get the finished files and try to prove the gold wrong.

→ findings.csv, with verdicts

11 Reconcile pathways

PATHWAYS · FLAGS

Matches your routes against `pathways.json`, one record per section; flags duplicates.

→ matched · missing · extra → findings.csv

2 Reconcile reactions

REACTIONS · FLAGS

Matches your list against `reactions.json` by identity, not by name or number.

→ matched · missing · extra → findings.csv

4 Reconcile compounds

COMPOUNDS · FLAGS

Matches your list against `compounds.json` through identifiers and aliases; flags duplicates.

→ matched · missing · extra → findings.csv

6 Verify compounds

COMPOUNDS · FLAGS

One record at a time, the whole `.md` re-read for each, every field checked.

→ compounds_audit.csv

8 Gold check compounds

COMPOUNDS · EDITS

Re-derives every field, corrects as it goes, merges duplicates, gives a verdict on every finding.

→ corrected file · yes / no

10 Identify pathways

PATHWAYS · FLAGS

Reads the `.md` alone and counts the distinct routes, from starting material to product.

→ your own route list and count

12 Gold check pathways

PATHWAYS · EDITS

Re-projects each step from the gold reactions and compounds, merges routes, checks every field.

→ pathways_audit.csv · yes / no

SECTION 6

Run order, step by step

Continue from step 5: at each step, paste the prompt named on the card into your Claude Code session, then move to the next. Stop at every gate.

